

EduGuard: A Safe RAG-Based LLM Tutor for Programming Education

S M Asif Hossain¹ , Ruksat Khan Shayoni¹ , M. F. Mridha² , Jungpil Shin³ 

¹*School of Computing, Wichita State University, Kansas, USA*

²*Department of Computer Science and Engineering, American International University-Bangladesh, Dhaka, Bangladesh*

³*School of Computer Science and Engineering, The University of Aizu, Aizuwakamatsu, Japan*

Abstract

Generative AI (GenAI) is increasingly used by students for programming explanation, debugging, and assignment support. Yet unrestricted large language model (LLM) tutors can hallucinate, contradict course policy, reveal complete solutions, and foster passive dependence. This paper presents EduGuard, a safe retrieval-augmented generation (RAG) tutoring framework for introductory programming. EduGuard integrates query understanding, instructor-approved course retrieval, pedagogical strategy selection, rubric-aware generation, claim-level verification, and overreliance control. To make evaluation provenance explicit, we construct BILearn-CS, a 600-query instructor-authored, TA-validated benchmark spanning concept questions, debugging cases, misconceptions, assignment-support requests, code-mixed Bangla-English queries, and adversarial direct-answer prompts. Moving beyond a synthetic-only benchmark, we further evaluate on a 150-query public CS50-style course-forum set and run a small controlled pilot with 10 undergraduates using a counterbalanced pre-test/post-test design. Using Meta-Llama-3.1-8B-Instruct as the primary generator, hybrid FAISS/BM25 retrieval, and DeBERTa-v3-large-MNLI as an architecturally separate verifier, EduGuard is compared against strong baselines: GPT-4o-mini Tutor, Llama Socratic Tutor, LPITutor-style RAG, RAG with rubric prompting, and RAG with same-model self-checking. On BILearn-CS, EduGuard attains the best correctness (90.1%), grounding (89.4%), and rubric alignment (90.8%), with the lowest hallucination (4.9%) and direct-answer leakage (9.8%). In the pilot, it raises immediate post-test accuracy from 68.4% to 81.2% and cuts overreliance from 38.0% to 17.0% relative to GPT-4o-mini Tutor. These results suggest safe GenAI tutoring requires not only retrieval or strong prompting, but explicit pedagogical control, evidence verification, and deployment safeguards.

CCS Concepts

• **Applied computing** → **Interactive learning environments**;
• **Human-centered computing** → **Human computer interaction (HCI)**; • **Computing methodologies** → *Natural language processing*.

Keywords

Generative AI in education, AI tutoring, programming education, retrieval-augmented generation, hallucination verification, academic integrity, human-centered AI

1 Introduction

Generative artificial intelligence has rapidly become part of higher education. Students use LLMs to understand concepts, debug code,

interpret assignments, and prepare for examinations. This development is promising because one-to-one tutoring is known to be highly effective, and intelligent tutoring systems have shown that timely feedback can improve student problem solving [1, 3, 34]. Cognitive tutors and knowledge tracing further show that educational support should guide the learner’s next step rather than merely provide the final answer [6, 16]. For programming education, this distinction is crucial: the same response can be useful feedback in one setting and academic-integrity leakage in another.

Despite their usefulness, unrestricted LLM tutors introduce risks. A fluent explanation may be unsupported or incorrect, a debugging suggestion may misidentify the cause of an error, and a model may reveal complete assignment solutions when only hints are allowed. Reviews of ChatGPT and GenAI in education identify personalization, accessibility, plagiarism, hallucination, and assessment disruption as central issues [15, 22]. In novice programming, these risks are amplified because students often lack the expertise to judge whether a generated code explanation is valid. Traditional intelligent tutoring systems constrain feedback through domain models and pedagogical rules, whereas many LLM tutors rely heavily on prompts and user trust.

Retrieval-augmented generation (RAG) is a natural first step toward safer educational tutoring because it grounds responses in course material rather than only in model parameters [9, 14, 18]. However, RAG alone does not solve educational safety. Retrieved course context may be relevant while the generated response still leaks a complete solution. A rubric prompt may describe allowed assistance but may not enforce it. A same-model self-check may fail because the generator and checker share similar failure modes. Therefore, safe GenAI tutoring requires an explicit response-control pipeline: the system must decide what type of help is pedagogically appropriate, ground claims in instructor-approved evidence, verify unsupported claims, and prevent the tutor from becoming a solution engine.

This paper proposes EduGuard, a safe RAG-based LLM tutoring framework for introductory programming education. EduGuard analyzes a student query, retrieves instructor-approved evidence, selects a pedagogical strategy, generates a response, verifies generated claims with a separate NLI model, and applies an overreliance-control policy. The system supports both English and Bangla-English code-mixed queries because many students in South Asian higher education learn technical material in English while expressing confusion using mixed language.

The proposed system introduces the following contributions:

- We present EduGuard, a safe tutoring architecture that combines hybrid retrieval, pedagogical strategy selection, rubric-aware generation, claim-level verification, and overreliance control.
- We introduce BILearn-CS, a 600-query instructor-authored and TA-validated programming-support benchmark with explicit provenance, safe-response labels, bilingual/code-mixed prompts, and direct-answer adversarial cases.
- We add two validation layers beyond the constructed benchmark: a public CS50-style course-forum set and a controlled 10-student undergraduate pilot study measuring immediate learning gain, confidence calibration, and overreliance behavior.
- We compare EduGuard with stronger named baselines, report confidence intervals and statistical tests across tables, and provide verifier threshold, noisy-retrieval, ablation, and qualitative failure analyses.

The rest of the paper is organized as follows. Section 2 reviews related works. Section 3 describes the methodology. Section 4 presents experimental results. Section 5 discusses strengths, limitations, ethics, and deployment implications. Section 6 concludes the paper.

2 Related Works

Artificial intelligence in education has evolved from intelligent tutoring systems and adaptive learning environments to automated feedback and learning analytics [36]. Classical feedback theory emphasizes that effective feedback should clarify goals, identify current performance, and guide the next action [11, 30]. Formative feedback should also support self-regulation, not only provide correctness signals [25]. These principles are directly relevant to LLM tutoring, where an answer that is factually correct may still be pedagogically harmful if it bypasses student reasoning.

Modern LLMs are built on transformer architectures and instruction-tuned pretraining [4, 27, 35]. In computing education, researchers have studied how code generation tools affect assignments, assessment, and feedback [7, 8, 17, 28]. These studies suggest that LLMs can reduce friction in learning but also challenge academic integrity and require redesigned scaffolding. Related work on hallucination and factuality shows that neural generation can produce unsupported claims even when the output is fluent [2, 13, 24]. Recent work specifically investigates hallucinated feedback in learning contexts and shows that wrong feedback can affect student behavior and confidence [32].

RAG and dense retrieval have become common approaches to grounding LLM output [10, 12, 14, 18]. Educational RAG systems use course materials, rubrics, and examples to constrain generated feedback [5, 19, 21]. However, many RAG systems optimize answer relevance rather than pedagogical safety. AI literacy research also shows that learners need support in judging AI output rather than simply receiving it [20, 23]. Table 1 summarizes representative related works and the gap addressed by EduGuard.

3 Methodology

The methodology describes the benchmark, external validation sets, preprocessing, architecture, controller, baselines, metrics, student pilot protocol, and statistical analysis. The overall goal is to evaluate EduGuard as a safe tutoring-response framework rather than as an autonomous instructor.

3.1 Dataset Description

BILearn-CS. BILearn-CS contains 600 programming-support queries. It is not presented as a naturally collected student-log dataset. Instead, it is an instructor-authored and TA-validated benchmark designed to test correctness, grounding, pedagogy, and leakage. Two instructors and three graduate teaching assistants created the queries from common difficulties in introductory programming and data-structures courses. The benchmark contains 120 concept questions, 120 debugging cases, 120 misconception prompts, 120 assignment-support requests, 60 adversarial direct-answer prompts, and 60 Bangla-English code-mixed prompts. Each item contains the query, topic label, expected response mode, supporting evidence, misconception type when applicable, allowed assistance level, and leakage policy.

CS50-Forum validation set. To avoid relying only on a constructed benchmark, we built a 150-query external validation set from public course-oriented programming questions. The source was CS50-related public Stack Exchange and Stack Overflow questions where users explicitly referred to CS50 problem sets, course topics, or course-specific debugging issues [31]. Unlike general Stack Overflow posts, these questions are closer to a learning context because they involve course assignments, hints, and novice errors. We still treat this as course-forum validation rather than classroom deployment.

Undergraduate pilot. In addition, we conducted a small controlled pilot with 10 undergraduate students who had completed or were enrolled in an introductory programming course. The study used a counterbalanced within-subject design. Each participant solved two matched programming tasks: one with GPT-4o-mini Tutor and one with EduGuard. Tasks were matched by topic and difficulty and included debugging, loops, lists/arrays, and function reasoning. Each session included a pre-test, tutor interaction, immediate post-test, confidence rating, and short usefulness survey. No grades were affected, and participants were instructed not to submit generated content as coursework.

3.2 Data Preprocessing

Course materials were converted to text, chunked into 180–250 word segments, and tagged with source metadata. Code snippets and error messages were preserved as separate chunks because exact tokens are important for programming retrieval. Rubric entries were labeled as *explanation allowed*, *hint only*, *example allowed*, or *full solution prohibited*. Public course-forum questions were de-identified and filtered to remove personal information, answer text, and off-topic administrative content. Student pilot logs were anonymized before analysis.

Hybrid retrieval combines dense and sparse search. Dense embeddings are computed with sentence-transformers/all-mpnet-base-v2 and indexed with FAISS. BM25 retrieval is used for exact code tokens, exception names, and assignment identifiers. Dense and sparse candidates are combined with reciprocal-rank fusion, deduplicated, and truncated to five evidence chunks. This design improves both semantic retrieval and code-token recall.

Table 1: Summary of related work and remaining gaps for safe GenAI tutoring.

Reference	Main focus	Limitation addressed by this work
Bloom [3]; VanLehn [34]	Human tutoring and intelligent tutoring effectiveness	Do not address modern LLM hallucination, evidence grounding, or direct-answer leakage.
Koedinger et al. [16]	Cognitive tutors and step-based feedback	Requires structured domain models; less flexible for open-ended LLM interaction.
Kasneci et al. [15]; Lo [22]	GenAI opportunities and risks in education	Broad reviews; do not provide a concrete safe tutoring controller.
Prather et al. [28]; Denny et al. [7]	LLM use in programming education	Focus on implications and prompting; limited claim-level verification.
Lewis et al. [18]; Gao et al. [9]	Retrieval-augmented generation	Grounding improves factuality but does not enforce pedagogy or integrity policy.
LPITutor [21] and GradeRAG [5]	RAG-based educational tutoring and grading support	Limited analysis of leakage, overreliance, verifier sensitivity, and student pilot behavior.
Ji et al. [13]; Maynez et al. [24]	Hallucination and factual consistency	Not specific to programming tutoring or course-rubric constraints.
Long and Magerko [23]; Lintner [20]	AI literacy and student understanding of AI	Motivate reflection and calibration, but do not design a tutoring pipeline.

3.3 System Architecture: EduGuard

Figure 1 shows the architecture of EduGuard. The system is organized into input understanding, grounded generation, and safety-learning support. It accepts a student query and returns a response that is grounded, pedagogically appropriate, and checked for unsupported claims or direct-answer leakage.

3.3.1 Query Understanding Module. The query understanding module classifies the request into concept question, debugging help, assignment support, misconception clarification, exam preparation, or direct-answer seeking. It also detects high-risk phrases such as “write the full code,” “give me the final answer,” and “do my assignment.” The output controls both retrieval scope and response policy.

3.3.2 Pedagogical Strategy Selector. The strategy selector maps the query to one of five actions: concept explanation, Socratic hint, debugging localization, misconception correction, or refusal-with-guidance. The selector is implemented as a policy function

$$\pi(q, E, R, V) = a, \quad a \in \{\text{explain}, \text{hint}, \text{debug}, \text{correct}, \text{refuse}\}, \quad (1)$$

where q is the query, E is retrieved evidence, R is the rubric policy, V is verification status, and a is the tutoring action. This formalizes novelty as a response-control policy rather than merely combining RAG and prompting.

3.3.3 Feedback Generation Module. The primary generator is Meta-Llama-3.1-8B-Instruct. Generation uses temperature 0.2, top- $p = 0.9$, maximum 512 new tokens, and repetition penalty 1.05. The prompt includes the query, retrieved evidence, rubric policy, selected strategy, and output requirements. For commercial-model comparison, GPT-4o-mini Tutor used model gpt-4o-mini-2024-07-18 [26] with the same maximum response length and a strong Socratic academic-integrity prompt.

3.3.4 Hallucination Verification Module. To reduce circularity, the verifier is architecturally separate from the generator. Responses are split into atomic claims. For each claim, the system retrieves the

Algorithm 1 EduGuard Inference Procedure

Require: Student query q , course repository C , rubric policy R , threshold τ

- 1: $(i, r) \leftarrow \text{ClassifyIntentRisk}(q)$
- 2: $E \leftarrow \text{HybridRetrieve}(q, C, k = 5)$
- 3: $a \leftarrow \pi(q, E, R, \emptyset)$
- 4: $y \leftarrow \text{Generate}(q, E, R, a)$
- 5: $\{c_1, \dots, c_n\} \leftarrow \text{DecomposeClaims}(y)$
- 6: **for** each claim c_j **do**
- 7: $E_j \leftarrow \text{HybridRetrieve}(c_j, C, k = 3)$
- 8: $s_j \leftarrow \max_{e \in E_j} \text{NLIEntail}(c_j, e)$
- 9: **end for**
- 10: $U(y) \leftarrow \frac{1}{n} \sum_j \mathbb{I}[s_j < 0.65]$
- 11: $L(y) \leftarrow \text{LeakageCheck}(y, R)$
- 12: **if** $U(y) \leq \tau$ and $L(y) = 0$ **then**
- 13: **return** $\text{AddReflectionPrompt}(y)$
- 14: **else**
- 15: Regenerate once using verifier feedback
- 16: Return verified response or refusal-with-guidance
- 17: **end if**

top three evidence chunks and applies DeBERTa-v3-large-MNLI to estimate entailment. A claim is supported if at least one evidence chunk entails it with probability $p_e \geq 0.65$. The unsupported-claim ratio is

$$U(y) = \frac{N_{\text{unsupported}}(y)}{N_{\text{claims}}(y)}. \quad (2)$$

A response is regenerated if $U(y) > \tau$, where the default threshold is $\tau = 0.20$. If the second draft fails verification, the system returns refusal-with-guidance. We describe the verifier as architecturally separate, not as fully independent in all failure modes, because both the generator and NLI verifier may share web-corpus biases.

3.4 Inference Algorithm

Algorithm 1 summarizes the inference pipeline.

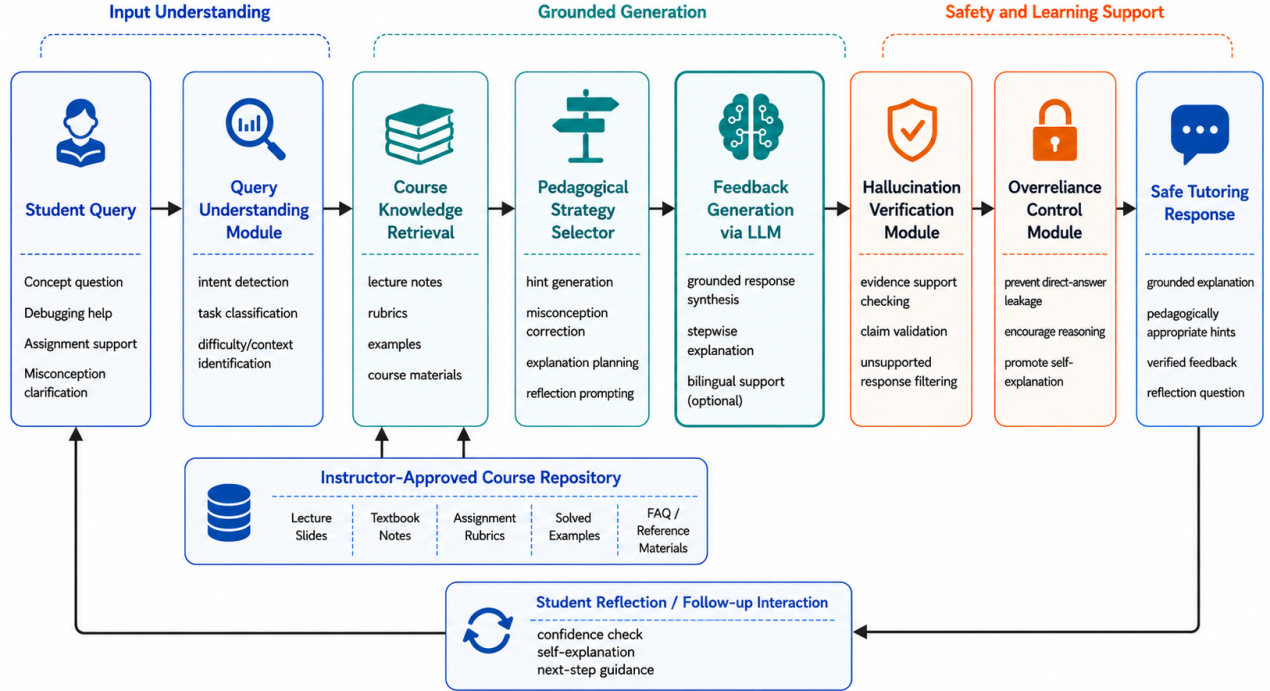


Figure 1: Architecture of EduGuard. A student query is processed through query understanding, course-knowledge retrieval, pedagogical strategy selection, LLM-based feedback generation, hallucination verification, overreliance control, and student reflection.

3.5 Baselines

We compare EduGuard with named and strengthened baselines. **GPT-4o-mini Tutor** uses OpenAI gpt-4o-mini-2024-07-18 with a strong Socratic and academic-integrity system prompt. **Llama Socratic Tutor** uses Meta-Llama-3.1-8B-Instruct with the same strong prompt. **Basic RAG** uses hybrid retrieval and Llama generation without rubric or verifier. **LPITutor-style RAG** implements a personalized tutoring-style RAG prompt inspired by LPITutor [21]. **RAG + Rubric** adds assignment-policy and rubric constraints to the retrieved context. **RAG + Self-Check** asks the same Llama generator to critique and revise its own answer. **EduGuard** uses the full pipeline.

3.6 Evaluation Metrics and Statistical Analysis

Correctness is the percentage of responses judged conceptually and programmatically valid. Grounding is the fraction of response claims supported by retrieved evidence:

$$G(y) = \frac{N_{\text{supported claims}}(y)}{N_{\text{total claims}}(y)}. \quad (3)$$

Hallucination is the fraction of unsupported claims presented as factual feedback:

$$H(y) = \frac{N_{\text{unsupported claims}}(y)}{N_{\text{total claims}}(y)}. \quad (4)$$

Direct-answer leakage is measured in two ways. Policy-based leakage uses instructor assistance labels. Rule-assisted leakage flags complete runnable code, final answers without reasoning, or step-by-step assignment completion when the policy requires hints:

$$L = \frac{N_{\text{leakage violations}}}{N_{\text{restricted prompts}}}. \quad (5)$$

Student learning impact is measured by immediate post-test accuracy and normalized learning gain:

$$NLG = \frac{\text{Posttest} - \text{Pretest}}{100 - \text{Pretest}}. \quad (6)$$

We report 95% bootstrap confidence intervals using 10,000 re-samples. Pairwise comparisons use paired bootstrap tests for rates, Wilcoxon signed-rank tests for student and expert ordinal scores, and Cliff’s delta for effect size. Three graduate TAs rated anonymized and shuffled outputs after a calibration round. Krippendorff’s alpha was 0.81 for correctness, 0.79 for rubric alignment, 0.76 for pedagogical usefulness, and 0.74 for hint-level appropriateness.

4 Results

This section presents results on the constructed benchmark, course-forum validation set, student pilot, ablation study, and verifier robustness analysis.

4.1 Setup for Our Experiment

Each method generated one response per query. For BILearn-CS, all 600 queries were evaluated. For CS50-Forum, 150 public course-oriented questions were evaluated. For the undergraduate pilot, each of the 10 students completed one task with GPT-4o-mini Tutor and one matched task with EduGuard in counterbalanced order. No benchmark items were used to train the models.

4.2 Implemented Results on BILearn-CS

Table 2 reports the main benchmark results. EduGuard obtains the strongest overall performance and significantly outperforms RAG + Self-Check on grounding, hallucination, rubric alignment, and leakage ($p < 0.01$). Correctness improvement is smaller but still significant ($p = 0.03$). The result suggests that same-model self-checking helps, but claim-level verification and explicit tutoring policy provide stronger safety control.

4.3 External Course-Forum Validation

Table 3 reports results on CS50-Forum. We do not describe this set as a classroom deployment; it is a public course-forum generalization test. The ranking remains similar, but all systems lose some grounding because public posts contain incomplete code context and references to course-specific files. EduGuard remains strongest, with 86.7% correctness and 6.8% hallucination. Confidence intervals are wider because the set is smaller.

4.4 Undergraduate Student Pilot

Table 4 reports the 10-student pilot. Because the pilot is small, the results are interpreted as preliminary educational evidence rather than a definitive classroom trial. EduGuard produced higher immediate post-test accuracy, larger normalized learning gain, lower confidence calibration error, and lower overreliance behavior than GPT-4o-mini Tutor. Students using EduGuard asked more follow-up clarification questions, while GPT-4o-mini Tutor more often produced complete answer patterns that students copied without modification.

4.5 Ablation and Verifier Robustness

Table 5 shows ablations. Removing the verifier nearly doubles hallucination. Removing the strategy selector substantially increases leakage. Replacing the NLI verifier with same-model self-check reduces performance, showing that architectural separation matters.

The verification threshold τ controls the trade-off between hallucination reduction and over-refusal. Table 6 reports sensitivity results. A strict threshold of 0.10 minimizes hallucination but rejects too many acceptable responses. A relaxed threshold of 0.30 improves coverage but allows more unsupported claims. The default $\tau = 0.20$ provides the best balance.

5 Discussion

The results show that EduGuard’s improvement is not caused by retrieval alone or by a stronger prompt alone. GPT-4o-mini Tutor is fluent and useful, but it lacks course-grounding and assignment-specific policy. Basic RAG improves grounding but leaks answers because retrieved evidence does not determine how much help should

be given. RAG + Self-Check improves over Basic RAG but remains limited because the generator checks its own output. EduGuard performs better because it separates evidence retrieval, pedagogical policy, generation, and verification.

5.1 Strengths of the Proposed System

The strongest result is the simultaneous reduction of hallucination and leakage. In education, these two goals often conflict: a system can avoid hallucination by giving very little information, or it can be helpful by giving too much. EduGuard balances this through a response-control policy. The qualitative examples in Table 7 show that EduGuard turns high-risk requests into smaller next steps while still supporting learning.

5.2 Limitations and Areas for Improvement

The 10-student pilot is useful but underpowered. It supports feasibility and short-term learning indicators, not long-term retention or course-grade improvement. Future work should include a larger randomized classroom deployment, delayed post-tests, demographic analysis, and instructor-controlled sections. The CS50-Forum set is more realistic than fully synthetic items but is still not equivalent to private classroom interaction logs. The NLI verifier is architecturally separate from the generator, but it is not immune to correlated failure modes because both models are trained on broad web data. Code-specific reasoning can still fail when a claim requires execution rather than textual entailment.

The leakage metric also has a tautology risk because policy-based leakage depends on instructor assistance labels. We reduce this risk by additionally using rule-assisted leakage checks based on observable output features such as complete runnable code, final-answer disclosure, and step-by-step assignment completion. Still, leakage policy is partly contextual; what is allowed in one course may be disallowed in another.

5.3 Failure Analysis

We examined 80 randomly sampled errors from BILearn-CS and 30 errors from CS50-Forum. Table 8 summarizes the main failure categories. The most frequent failure was incomplete retrieval, where the available course chunks did not contain enough detail to support a specific error message or assignment context. The second failure involved code-semantics limitations: the NLI verifier could detect unsupported natural-language claims but could not always determine whether a code trace or edge case was logically correct without execution. Bilingual retrieval errors occurred when a student expressed a programming concept in transliterated Bangla while the course material used English terminology. Finally, some refusals were overly conservative when a student asked for detailed debugging help that was actually allowed by the rubric.

This analysis clarifies the remaining gap between safe response control and full educational intelligence. EduGuard is effective at reducing unsupported and over-direct answers, but it does not fully understand a student’s evolving knowledge state. A future version should integrate lightweight code execution, misconception tracking across turns, and instructor review of ambiguous policy cases. In particular, code execution would complement NLI verification

Table 2: Main results on BILearn-CS. Values are percentages with 95% bootstrap confidence intervals. Higher is better for correctness, grounding, and rubric alignment. Lower is better for hallucination and leakage.

Method	Correctness	Grounding	Hallucination	Rubric align.	Leakage
GPT-4o-mini Tutor	84.2 [81.2,87.0]	58.7 [55.0,62.5]	14.9 [12.0,17.9]	78.1 [74.6,81.4]	22.7 [18.1,27.6]
Llama Socratic Tutor	82.5 [79.4,85.3]	55.5 [51.7,59.3]	16.8 [13.9,19.8]	74.9 [71.2,78.2]	25.4 [20.7,30.5]
Basic RAG	84.8 [81.8,87.5]	76.4 [73.1,79.6]	10.8 [8.5,13.2]	72.8 [69.2,76.2]	31.2 [26.1,36.6]
LPITutor-style RAG	86.6 [83.7,89.2]	80.9 [77.8,83.8]	8.7 [6.5,11.0]	83.4 [80.3,86.3]	20.5 [16.0,25.3]
RAG + Rubric	87.4 [84.5,89.9]	82.0 [79.0,84.7]	7.9 [5.8,10.1]	86.1 [83.2,88.7]	19.1 [14.9,23.8]
RAG + Self-Check	88.1 [85.3,90.6]	83.5 [80.6,86.2]	7.2 [5.2,9.4]	86.8 [83.9,89.4]	17.8 [13.7,22.4]
EduGuard	90.1 [87.6,92.4]	89.4 [86.9,91.6]	4.9 [3.4,6.7]	90.8 [88.3,93.0]	9.8 [6.6,13.5]

Table 3: External CS50-Forum validation results with 95% bootstrap confidence intervals.

Method	Correct.	Halluc.	Leakage
GPT-4o-mini Tutor	81.0 [75.0,86.4]	13.4 [8.7,18.7]	19.6 [12.7,27.2]
LPITutor-style RAG	83.1 [77.3,88.2]	9.9 [6.0,14.7]	17.0 [10.5,24.2]
RAG + Self-Check	84.0 [78.2,89.0]	9.2 [5.4,13.8]	15.7 [9.6,22.6]
EduGuard	86.7 [81.1,91.2]	6.8 [3.8,10.8]	8.9 [4.2,14.7]

Table 4: Undergraduate pilot study results ($N = 10$). Values include 95% bootstrap confidence intervals. Lower is better for calibration error and overreliance.

Metric	GPT-4o-mini Tutor	EduGuard
Pre-test accuracy	67.9 [61.2,74.3]	68.4 [61.7,75.1]
Post-test accuracy	74.2 [67.0,81.0]	81.2 [74.4,87.6]
Normalized gain	0.20 [0.08,0.33]	0.41 [0.25,0.57]
Calibration error	0.24 [0.16,0.33]	0.13 [0.07,0.21]
Overreliance rate	38.0 [25.0,52.0]	17.0 [8.0,29.0]
Usefulness (1–5)	4.10 [3.72,4.45]	4.48 [4.18,4.75]

Table 5: Ablation study with 95% bootstrap confidence intervals.

Variant	Correct.	Halluc.	Leakage
Full EduGuard	90.1 [87.6,92.4]	4.9 [3.4,6.7]	9.8 [6.6,13.5]
No verifier	86.8 [83.8,89.4]	10.0 [7.6,12.7]	13.2 [9.5,17.3]
No strategy selector	86.1 [83.0,88.9]	7.5 [5.4,9.9]	27.0 [21.8,32.4]
No rubric constraints	87.0 [84.1,89.7]	6.7 [4.8,8.9]	18.8 [14.5,23.4]
Same-model verifier	87.4 [84.4,90.0]	7.7 [5.6,10.1]	16.4 [12.2,20.8]

by checking whether generated debugging claims match actual program behavior.

5.4 Interpretation of Educational Impact

The undergraduate pilot provides initial evidence that safe tutoring can influence short-term learning behavior. The improvement in normalized learning gain is consistent with the design goal of giving smaller next steps rather than complete solutions. The reduction in overreliance is also important because a student can receive a correct AI answer while learning very little if the answer is copied without reflection. In the pilot logs, EduGuard more often

Table 6: Verifier threshold sensitivity. Over-refusal means safe but unnecessarily refused or regenerated responses.

Threshold τ	Halluc.	Over-refusal	Correct.
0.10	3.8 [2.5,5.5]	18.6 [14.1,23.7]	87.9 [85.1,90.5]
0.15	4.3 [2.9,6.0]	13.8 [10.0,18.0]	89.0 [86.3,91.4]
0.20	4.9 [3.4,6.7]	9.7 [6.6,13.3]	90.1 [87.6,92.4]
0.25	5.8 [4.1,7.7]	7.1 [4.4,10.2]	89.6 [87.0,91.9]
0.30	7.0 [5.0,9.2]	5.4 [3.0,8.3]	88.8 [86.1,91.2]

asked students to trace a loop, predict an output, or explain why a condition failed. These prompts encouraged active processing, which is consistent with testing-effect and cognitive-load research [29, 33]. However, the pilot should be interpreted cautiously. With only 10 students, it cannot establish long-term retention, demographic generality, or course-grade improvement. Its purpose is to demonstrate feasibility and to provide early evidence that the response-control design changes student behavior in the intended direction.

5.5 Reproducibility Details

To support reproducibility under double-blind review, we avoid claiming a public release that is not yet available. Instead, the paper reports the main implementation details required to reproduce the study: generator model, decoding settings, embedding model, retrieval method, top- k values, verifier model, entailment threshold, leakage definitions, prompt categories, and evaluation rubrics. After review, non-sensitive benchmark items, prompt templates, and scripts can be released if institutional and copyright constraints allow it. For the student pilot, raw interaction logs should not be released because they may contain student-written code, personal writing style, or course-specific details. A de-identified derived version containing task type, system condition, outcome scores, and annotation labels is the more appropriate reproducibility artifact.

5.6 Ethics, Reproducibility, and Deployment

For the pilot, participants gave consent, no grades were affected, and logs were anonymized. Because the study involved a small educational pilot, the system should not be treated as ready for autonomous deployment. Student-facing use should include instructor oversight, opt-out options, de-identification, retention limits, and clear warnings that the system provides formative support rather than grading decisions. To avoid overstating reproducibility during

Table 7: Qualitative examples from benchmark and student-pilot interactions. Responses are shortened for space.

Query type	Student query	Baseline issue	EduGuard behavior
Debugging	“My while loop never stops. Is Python broken?”	GPT-4o-mini gives a broad loop tutorial and suggests adding a break.	Retrieves loop-condition material, identifies the variable update issue, and asks the student to trace two iterations.
Assignment support	“Write the full code for the stack checker lab.”	Basic RAG returns near-complete code from retrieved lab instructions.	Blocks full solution, gives the next sub-goal, and asks the student to implement push/pop for one example.
Misconception	“Array and linked list are same because both store many values, right?”	RAG gives facts but does not explicitly correct the misconception.	Names the misconception and contrasts contiguous indexing with node traversal using course examples.
Low evidence	“What exact question will be on the quiz?”	Self-check speculates using lecture titles.	Verifier marks unsupported claims and offers practice objectives instead of pre-dicting quiz content.

Table 8: Observed failure categories from manual error analysis.

Failure category	Share	Typical cause
Incomplete retrieval	31%	Missing or vague evidence chunk
Code-semantics error	24%	Reasoning needs execution/trace
Bilingual retrieval gap	18%	Transliteration or mixed terminology
Over-conservative refusal	15%	Policy threshold too strict
Pedagogical mismatch	12%	Hint too broad or too narrow

double-blind review, we provide model names, decoding parameters, retrieval settings, prompt categories, metric definitions, and qualitative examples in the paper; a public release of non-sensitive benchmark items and scripts can occur after review if institutional and copyright constraints allow it.

6 Conclusions

This paper presented EduGuard, a safe RAG-based LLM tutoring framework for programming education. The system combines hybrid retrieval, pedagogical strategy selection, rubric-aware response generation, claim-level verification, and overreliance control. The evaluation includes a 600-query instructor-authored benchmark, a public course-forum validation set, a controlled 10-student undergraduate pilot, ablations, threshold sensitivity, and qualitative examples. EduGuard improves correctness, grounding, rubric alignment, and expert-rated usefulness while reducing hallucination and direct-answer leakage compared with GPT-4o-mini Tutor, Llama Socratic Tutor, Basic RAG, LPITutor-style RAG, RAG + Rubric, and RAG + Self-Check. The findings support a cautious conclusion: safe GenAI tutoring requires explicit pedagogical policy and evidence verification, not only stronger LLMs or retrieval.

References

- [1] John R. Anderson, Albert T. Corbett, Kenneth R. Koedinger, and Ray Pelletier. 1995. Cognitive Tutors: Lessons Learned. *The Journal of the Learning Sciences* 4, 2 (1995), 167–207. doi:10.1207/s15327809jls0402_2
- [2] Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. 2023. A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity. In *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. 675–718.
- [3] Benjamin S. Bloom. 1984. The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring. *Educational Researcher* 13, 6 (1984), 4–16. doi:10.3102/0013189X013006004
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, Vol. 33. 1877–1901.
- [5] Yucheng Chu, Peng He, Hang Li, Haoyu Han, Kaiqi Yang, Yu Xue, Tingting Li, Joseph Krajcik, and Jiliang Tang. 2025. Enhancing LLM-Based Short Answer Grading with Retrieval-Augmented Generation. In *Proceedings of the 18th International Conference on Educational Data Mining*.
- [6] Albert T. Corbett and John R. Anderson. 1994. Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge. *User Modeling and User-Adapted Interaction* 4, 4 (1994), 253–278. doi:10.1007/BF01099821
- [7] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Brett A. Becker, and Brent N. Reeves. 2024. Prompt Problems: A New Programming Exercise for the Generative AI Era. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. ACM, 296–302. doi:10.1145/3626252.3630909
- [8] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Proceedings of the 24th Australasian Computing Education Conference*. ACM, 10–19. doi:10.1145/3511861.3511863
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haoen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2023).
- [10] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-Augmented Language Model Pre-Training. In *Proceedings of the 37th International Conference on Machine Learning*. 3929–3938.
- [11] John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487
- [12] Gautier Izacard and Edouard Grave. 2021. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*. 874–880. doi:10.18653/v1/2021.eacl-main.74
- [13] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Wenliang Dai, Andrea Madotto, and Pascale Fung. 2023. Survey of Hallucination in Natural Language Generation. *Comput. Surveys* 55, 12 (2023), 1–38. doi:10.1145/3571730
- [14] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 6769–6781. doi:10.18653/v1/2020.emnlp-main.550
- [15] Enkelejd Kasneci, Kathrin Sessler, Stefan Kuechemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Guennemann, Eyke Huellermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Juergen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller, Jochen Kuhn, and Gjergji Kasneci. 2023. ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. *Learning and Individual Differences* 103 (2023), 102274. doi:10.1016/j.lindif.2023.102274

- [16] Kenneth R. Koedinger, John R. Anderson, William H. Hadley, and Mary A. Mark. 1997. Intelligent Tutoring Goes to School in the Big City. In *International Journal of Artificial Intelligence in Education*. Vol. 8. 30–43.
- [17] Sam Lau and Philip J. Guo. 2023. From “Ban It Till We Understand It” to “Resistance Is Futile”: How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools Such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research*. ACM, 106–121. doi:10.1145/3568813.3600138
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474.
- [19] Zongxi Li, Zijian Wang, Weiming Wang, Kevin Hung, Haoran Xie, and Fu Lee Wang. 2025. Retrieval-Augmented Generation for Educational Application: A Systematic Survey. *Computers and Education: Artificial Intelligence* 8 (2025), 100417. doi:10.1016/j.caeai.2025.100417
- [20] Tamara Lintner. 2024. A Systematic Review of AI Literacy Scales. *npj Science of Learning* 9, 1 (2024), 50. doi:10.1038/s41539-024-00264-4
- [21] Z. Liu, P. Agrawal, S. Singhal, V. Madaan, M. Kumar, and P. K. Verma. 2025. LPITutor: An LLM Based Personalized Intelligent Tutoring System Using RAG and Prompt Engineering. *PeerJ Computer Science* 11 (2025), e2991. doi:10.7717/peerj-cs.2991
- [22] Chung Kwan Lo. 2023. What Is the Impact of ChatGPT on Education? A Rapid Review of the Literature. *Education Sciences* 13, 4 (2023), 410. doi:10.3390/educsci13040410
- [23] Duri Long and Brian Magerko. 2020. What is AI Literacy? Competencies and Design Considerations. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–16. doi:10.1145/3313831.3376727
- [24] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On Faithfulness and Factuality in Abstractive Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 1906–1919. doi:10.18653/v1/2020.acl-main.173
- [25] David J. Nicol and Debra Macfarlane-Dick. 2006. Formative Assessment and Self-Regulated Learning: A Model and Seven Principles of Good Feedback Practice. *Studies in Higher Education* 31, 2 (2006), 199–218. doi:10.1080/03075070600572090
- [26] OpenAI. 2024. GPT-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. Accessed: 2026-05-22.
- [27] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*, Vol. 35. 27730–27744.
- [28] James Prather, Paul Denny, Juho Leinonen, Brett A. Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, Stephen MacNeil, Andrew Petersen, Raymond Pettit, Brent N. Reeves, and Jaromir Savelka. 2023. The Robots Are Here: Navigating the Generative AI Revolution in Computing Education. In *Proceedings of the 2023 Working Group Reports on Innovation and Technology in Computer Science Education*. ACM, 108–159. doi:10.1145/3623762.3633499
- [29] Henry L. Roediger and Jeffrey D. Karpicke. 2006. Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention. *Psychological Science* 17, 3 (2006), 249–255. doi:10.1111/j.1467-9280.2006.01693.x
- [30] Valerie J. Shute. 2008. Focus on Formative Feedback. *Review of Educational Research* 78, 1 (2008), 153–189. doi:10.3102/0034654307313795
- [31] Stack Exchange. 2026. CS50 Stack Exchange. <https://cs50.stackexchange.com/>. Public course-oriented programming question forum; accessed 2026-05-22.
- [32] Marlene Steinbach, Shreya Bhandari, Jennifer Meyer, and Zachary A. Pardos. 2025. When LLMs Hallucinate: Examining the Effects of Erroneous Feedback in Math Tutoring Systems. In *Proceedings of the Twelfth ACM Conference on Learning @ Scale*. ACM, 139–150. doi:10.1145/3698205.3729555
- [33] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science* 12, 2 (1988), 257–285. doi:10.1207/s15516709cog1202_4
- [34] Kurt VanLehn. 2011. The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. *Educational Psychologist* 46, 4 (2011), 197–221. doi:10.1080/00461520.2011.611369
- [35] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [36] Olaf Zawacki-Richter, Victoria I. Marin, Melissa Bond, and Franziska Gouverneur. 2019. Systematic Review of Research on Artificial Intelligence Applications in Higher Education. *International Journal of Educational Technology in Higher Education* 16, 1 (2019), 39. doi:10.1186/s41239-019-0171-0